

APOSTILA DO ALUNO

Programação Web II

Etec – Ensino Técnico

Técnico em Desenvolvimento de Sistemas

2026

Sumário

Capítulo 1 – Introdução aos Frameworks Front-End .. 3

- 1.1 Renderização no lado do cliente 3
- 1.2 Reatividade e interação com o usuário 5
- 1.3 Componentes 7
- 1.4 Renderização condicional e de listas 10
- 1.5 Eventos 13
- 1.6 SPAs, rotas e navegação 15
- 1.7 Estado e gerenciamento de estado 18

Capítulo 2 – Frameworks CSS 21

- 2.1 Leiaute e sistema de grade 21
- 2.2 Componentes CSS 24
- 2.3 Ícones 26
- 2.4 Customização 27

Capítulo 3 – Práticas de Desenvolvimento de Sistemas 29

- 3.1 Testes unitários 29
- 3.2 Testes de interface 32
- 3.3 Builds e releases 34
- 3.4 Controle de versão 36

Capítulo 1 – Introdução aos Frameworks Front-End

Neste capítulo, estudaremos os fundamentos dos frameworks front-end modernos, com ênfase na construção de interfaces web dinâmicas e reativas. Abordaremos conceitos como renderização no lado do cliente, componentes, eventos, roteamento e gerenciamento de estado.

1.1 Renderização no lado do cliente

Tradicionalmente, as páginas web eram renderizadas no servidor: o servidor processava o HTML e enviava o conteúdo pronto para o navegador. Com o avanço do JavaScript e o surgimento de frameworks modernos, a renderização passou a ser feita também no lado do cliente (client-side rendering).

O que é renderização no lado do cliente?

Na renderização cliente-side, o navegador recebe um arquivo HTML mínimo contendo uma tag raiz e um link para um arquivo JavaScript. Esse JavaScript é executado e constrói toda a interface do usuário dinamicamente no navegador.

Vantagens

- Navegação mais rápida após o carregamento inicial, pois apenas dados são trafegados.
- Experiência de usuário mais fluida, sem recarregamentos completos de página.
- Menor carga no servidor, que atua apenas como fornecedor de dados (API).

Desvantagens

- Tempo de carregamento inicial maior (mais JavaScript para baixar e executar).
- SEO mais desafiador, exigindo técnicas como SSR (Server-Side Rendering).
- Maior consumo de recursos do dispositivo do cliente.

Exemplo prático

Considere o seguinte código React que renderiza uma mensagem na tela:

```
// index.html
<div id="root"></div>
<script src="app.js"></script>

// app.jsx
import React from 'react';
import ReactDOM from 'react-dom/client';

function App() {
  return <h1>Olá, mundo!</h1>;
}

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<App />);
```

O navegador carrega o HTML, baixa o JavaScript e, em seguida, o React constrói a árvore de componentes e a insere no DOM.

Dica: O React utiliza o Virtual DOM para otimizar as atualizações da interface. Em vez de manipular o DOM real a cada alteração, o React atualiza primeiro uma representação virtual e depois calcula a forma mais eficiente de aplicar as mudanças.

1.2 Reatividade e interação com o usuário

Um dos pilares dos frameworks front-end modernos é a reatividade. Reatividade significa que a interface do usuário é atualizada automaticamente sempre que o estado subjacente dos dados é modificado.

Conceito de reatividade

Em um modelo reativo, os componentes "observam" os dados. Quando um dado muda, todos os componentes que dependem dele são automaticamente renderizados novamente. Isso elimina a necessidade de manipulação manual do DOM.

Exemplo sem reatividade (JavaScript puro)

```
let contador = 0;
const elemento = document.getElementById('contador');

function incrementar() {
  contador++;
  elemento.textContent = contador; // Manipulação manual do DOM
}
```

Exemplo com reatividade (React)

```
import React, { useState } from 'react';

function Contador() {
  const [contador, setContador] = useState(0);

  function incrementar() {
    setContador(contador + 1); // React atualiza a UI automaticamente
  }

  return (
    <div>
      <p>Valor: {contador}</p>
      <button onClick={incrementar}>Incrementar</button>
    </div>
  );
}
```

Note que, no exemplo com React, não precisamos manipular o DOM diretamente. O React detecta a mudança de estado e re-renderiza apenas a parte necessária da interface.

O ciclo de atualização

O ciclo de atualização em um framework reativo segue estas etapas:

- O usuário interage com a interface (clique, digitação, etc.).
- Um evento é disparado e o estado é atualizado.
- O framework detecta a mudança de estado.
- O componente é re-renderizado com o novo estado.
- O DOM é atualizado de forma eficiente (via Virtual DOM, no caso do React).

Dica: No Vue.js, a reatividade é implementada por meio de Proxies (Vue 3) ou getters/setters (Vue 2). No React, o modelo é baseado em estado imutável e hooks como `useState` e `useReducer`.

1.3 Componentes

Componentes são a unidade fundamental de construção de interfaces em frameworks front-end modernos. Um componente é uma peça reutilizável e independente que encapsula estrutura (HTML), estilo (CSS) e comportamento (JavaScript).

O que é um componente?

Um componente pode ser comparado a uma "peça de Lego" digital. Cada componente tem sua própria responsabilidade e pode ser combinado com outros componentes para formar interfaces complexas.

Componentes funcionais no React

Atualmente, a forma recomendada de criar componentes em React é por meio de funções JavaScript que retornam JSX (JavaScript XML).

```
// Componente funcional simples
function Saudacao(props) {
  return <h1>Olá, {props.nome}!</h1>;
}

// Uso do componente
function App() {
  return (
    <div>
      <Saudacao nome="Maria" />
      <Saudacao nome="João" />
    </div>
  );
}
```

Props (Propriedades)

Props são os dados que um componente pai passa para um componente filho. Elas são somente leitura (imutáveis) e permitem a personalização dos componentes.

```
function Card(props) {
  return (
    <div className="card">
      <h2>{props.titulo}</h2>
      <p>{props.descricao}</p>
    </div>
  );
}

// Utilizando o componente Card com props diferentes
<Card titulo="React" descricao="Framework front-end" />
<Card titulo="Vue" descricao="Framework progressivo" />
```

Composição de componentes

A composição é o ato de combinar componentes menores para formar componentes maiores e mais complexos. Isso segue o princípio de que cada componente deve ter uma única responsabilidade.

```
function Avatar(props) {  
  return <img src={props.foto} alt={props.nome} className="avatar" />;  
}  
  
function InfoUsuario(props) {  
  return (  
    <div>  
      <h3>{props.nome}</h3>  
      <p>{props.email}</p>  
    </div>  
  );  
}  
  
function PerfilUsuario(props) {  
  return (  
    <div className="perfil">  
      <Avatar foto={props.foto} nome={props.nome} />  
      <InfoUsuario nome={props.nome} email={props.email} />  
    </div>  
  );  
}
```

Dica: Sempre mantenha seus componentes pequenos e focados. Um componente que faz muitas coisas deve ser dividido em componentes menores e mais especializados.

1.4 Renderização condicional e de listas

Em aplicações web dinâmicas, frequentemente precisamos exibir ou ocultar elementos com base em condições, ou renderizar listas de dados. Os frameworks front-end oferecem maneiras declarativas de fazer isso.

Renderização condicional

A renderização condicional permite exibir diferentes elementos dependendo do estado da aplicação.

Usando o operador &&

```
function Mensagem({ temNotificacao }) {  
  return (  
    <div>  
      <h1>Bem-vindo!</h1>  
      {temNotificacao && <p>Você tem novas mensagens.</p>}  
    </div>  
  );  
}
```

Usando operador ternário

```
function Status({ logado }) {  
  return (  
    <div>  
      {logado ? <p>Usuário logado</p> : <p>Faça login</p>}  
    </div>  
  );  
}
```

Usando if/else

```
function Painel({ usuario }) {  
  if (!usuario) {  
    return <p>Carregando...</p>;  
  }  
  if (usuario.nivel === 'admin') {  
    return <PainelAdmin usuario={usuario} />;  
  }  
  return <PainelComum usuario={usuario} />;  
}
```

Renderização de listas

Para renderizar listas de dados, utilizamos o método `map()` dos arrays JavaScript.

```
function ListaTarefas() {  
  const tarefas = [  
    { id: 1, texto: 'Estudar React' },  
    { id: 2, texto: 'Fazer exercícios' },  
    { id: 3, texto: 'Revisar conteúdo' }  
  ];  
  
  return (  
    <ul>  
      {tarefas.map((tarefa) => (  

```

```

        <li key={tarefa.id}>{tarefa.texto}</li>
      ))}
    </ul>
  );
}

```

É fundamental fornecer uma prop **key** única para cada item da lista. Isso ajuda o React a identificar quais itens foram alterados, adicionados ou removidos, otimizando a renderização.

Filtrando listas

```

function TarefasConcluidas({ tarefas }) {
  const concluidas = tarefas.filter((t) => t.concluida);

  return (
    <ul>
      {concluidas.map((t) => (
        <li key={t.id}>{t.texto}</li>
      ))}
    </ul>
  );
}

```

Dica: Evite usar o índice do array como key quando a lista for dinâmica (com inserções, remoções ou reordenações). Use um identificador único e estável, como um ID.

1.5 Eventos

Eventos são ações ou ocorrências que acontecem no navegador, como cliques, teclas pressionadas, envio de formulários, movimentação do mouse, entre outros. Nos frameworks front-end, os eventos são tratados de forma declarativa.

Eventos em React

Em React, os eventos são nomeados usando camelCase (onClick, onChange, onSubmit) e recebem funções como manipuladores.

```
function BotaoCurtir() {
  function handleClick() {
    alert('Você curtiu!');
  }

  return <button onClick={handleClick}>Curtir</button>;
}
```

Eventos com parâmetros

Quando precisamos passar parâmetros para o manipulador de eventos, utilizamos uma função arrow:

```
function ListaUsuarios({ usuarios }) {
  function saudar(nome) {
    alert(`Olá, ${nome}!`);
  }

  return (
    <ul>
      {usuarios.map((u) => (
        <li key={u.id}>
          {u.nome}
          <button onClick={() => saudar(u.nome)}>Saudar</button>
        </li>
      ))}
    </ul>
  );
}
```

Evento de formulário

```
function FormularioLogin() {
  const [email, setEmail] = useState('');
  const [senha, setSenha] = useState('');

  function handleSubmit(evento) {
    evento.preventDefault(); // Evita recarregar a página
    console.log('Login:', email, senha);
  }

  return (
    <form onSubmit={handleSubmit}>
      <input
        type="email"
        value={email}
        onChange={(e) => setEmail(e.target.value)}
      />
    </form>
  );
}
```

```

        placeholder="E-mail"
      />
      <input
        type="password"
        value={senha}
        onChange={(e) => setSenha(e.target.value)}
        placeholder="Senha"
      />
      <button type="submit">Entrar</button>
    </form>
  );
}

```

O método **preventDefault()** é usado para evitar o comportamento padrão do navegador, como recarregar a página ao submeter um formulário.

Evento de teclado

```

function Busca() {
  function handleKeyDown(evento) {
    if (evento.key === 'Enter') {
      console.log('Buscar:', evento.target.value);
    }
  }

  return (
    <input
      type="text"
      onKeyDown={handleKeyDown}
      placeholder="Pressione Enter para buscar"
    />
  );
}

```

Dica: Em React, você não precisa usar `addEventListener`. Basta passar a função diretamente como `prop` do elemento `JSX`. O React gerencia a adição e remoção dos listeners automaticamente.

1.6 SPAs, rotas e navegação

Aplicações de Página Única (Single Page Applications - SPAs) são aplicações web que carregam uma única página HTML e atualizam dinamicamente o conteúdo conforme o usuário interage, sem recarregar a página.

O que é uma SPA?

Em uma SPA, todo o código (HTML, CSS, JavaScript) é carregado uma única vez. A partir daí, apenas dados são trafegados entre o cliente e o servidor, geralmente em formato JSON. A navegação entre "páginas" é feita via JavaScript, manipulando a URL e o histórico do navegador.

React Router DOM

O React Router é a biblioteca padrão para roteamento em aplicações React. Ela permite definir rotas e navegar entre diferentes componentes.

Instalação

```
npm install react-router-dom
```

Configuração básica

```
import { BrowserRouter, Routes, Route, Link } from 'react-router-dom';

function Home() { return <h1>Página Inicial</h1>; }
function Sobre() { return <h1>Sobre Nós</h1>; }
function Contato() { return <h1>Contato</h1>; }

function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to="/">Home</Link>
        <Link to="/sobre">Sobre</Link>
        <Link to="/contato">Contato</Link>
      </nav>

      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/sobre" element={<Sobre />} />
        <Route path="/contato" element={<Contato />} />
      </Routes>
    </BrowserRouter>
  );
}
```

O componente **BrowserRouter** gerencia o histórico de navegação. **Routes** e **Route** definem quais componentes renderizar para cada URL. **Link** substitui a tag <a> do HTML, evitando o recarregamento da página.

Rotas dinâmicas

```
import { useParams } from 'react-router-dom';

function Produto() {
  const { id } = useParams();
  return <h1>Produto: {id}</h1>;
}
```

```
}

// Rota definida como: <Route path="/produto/:id" element={<Produto />} />
// URL: /produto/123 -> renderiza "Produto: 123"
```

Navegação programática

```
import { useNavigate } from 'react-router-dom';

function Login() {
  const navigate = useNavigate();

  function handleLogin() {
    // Lógica de autenticação...
    navigate('/dashboard'); // Redireciona após o login
  }

  return <button onClick={handleLogin}>Entrar</button>;
}
```

Dica: *Vantagens das SPAs: navegação instantânea, transições suaves e menos tráfego de dados. Desvantagens: SEO mais complexo e maior tempo de carregamento inicial.*

1.7 Estado e gerenciamento de estado

O estado de uma aplicação representa os dados que podem mudar ao longo do tempo e que afetam o que é exibido na tela. Gerenciar o estado é uma das tarefas mais importantes no desenvolvimento de aplicações front-end.

Estado local com useState

O hook **useState** permite adicionar estado a componentes funcionais.

```
import React, { useState } from 'react';

function Contador() {
  const [contador, setContador] = useState(0);

  return (
    <div>
      <p>Contagem: {contador}</p>
      <button onClick={() => setContador(contador + 1)}>+</button>
      <button onClick={() => setContador(contador - 1)}>-</button>
      <button onClick={() => setContador(0)}>Zerar</button>
    </div>
  );
}
```

useState retorna um array com dois elementos: o valor atual do estado e uma função para atualizá-lo.

Estado em objetos

```
function Perfil() {
  const [usuario, setUsuario] = useState({
    nome: '',
    email: '',
    idade: 0
  });

  function atualizarCampo(campo, valor) {
    setUsuario({ ...usuario, [campo]: valor });
  }

  return (
    <div>
      <input
        value={usuario.nome}
        onChange={(e) => atualizarCampo('nome', e.target.value)}
        placeholder="Nome"
      />
      <input
        value={usuario.email}
        onChange={(e) => atualizarCampo('email', e.target.value)}
        placeholder="E-mail"
      />
      <p>{usuario.nome} - {usuario.email}</p>
    </div>
  );
}
```

Elevação de estado (Lifting State Up)

Quando dois ou mais componentes precisam compartilhar o mesmo estado, elevamos o estado ao ancestral comum mais próximo.

```
function Termometro({ temperatura, escala }) {
  if (escala === 'c') {
    return <p>{temperatura}°C</p>;
  }
  return <p>{(temperatura * 9/5) + 32}°F</p>;
}

function ConversorTemperatura() {
  const [temperatura, setTemperatura] = useState(0);
  const [escala, setEscala] = useState('c');

  return (
    <div>
      <input
        type="number"
        value={temperatura}
        onChange={(e) => setTemperatura(Number(e.target.value))}
      />
      <select onChange={(e) => setEscala(e.target.value)}>
        <option value="c">Celsius</option>
        <option value="f">Fahrenheit</option>
      </select>
      <Termometro temperatura={temperatura} escala={escala} />
    </div>
  );
}
```

useEffect e efeitos colaterais

```
import React, { useState, useEffect } from 'react';

function DadosExternos() {
  const [dados, setDados] = useState([]);
  const [carregando, setCarregando] = useState(true);

  useEffect(() => {
    fetch('https://api.exemplo.com/dados')
      .then((res) => res.json())
      .then((dados) => {
        setDados(dados);
        setCarregando(false);
      });
  }, []); // Array vazio = executa apenas na montagem

  if (carregando) return <p>Carregando...</p>;

  return (
    <ul>
      {dados.map((item) => (
        <li key={item.id}>{item.nome}</li>
      ))}
    </ul>
  );
}
```

Dica: Dica importante: nunca modifique o estado diretamente (ex: `estado.push(novoItem)`). Sempre use a função de atualização (`setEstado`) e trabalhe com imutabilidade, criando novos arrays/objetos.

Capítulo 2 – Frameworks CSS

Frameworks CSS são conjuntos de estilos pré-definidos que aceleram o desenvolvimento de interfaces web. Eles oferecem componentes prontos, sistemas de grade responsivos e utilitários de customização.

2.1 Leiaute e sistema de grade

O sistema de grade (grid system) é uma das funcionalidades mais importantes dos frameworks CSS. Ele permite criar layouts responsivos de forma rápida e consistente.

Bootstrap Grid

O Bootstrap utiliza um sistema de grade baseado em flexbox, com 12 colunas. A linha (row) contém colunas (col) que se ajustam automaticamente.

```
<!-- Grid de 3 colunas iguais -->
<div class="container">
  <div class="row">
    <div class="col">
      Coluna 1
    </div>
    <div class="col">
      Coluna 2
    </div>
    <div class="col">
      Coluna 3
    </div>
  </div>
</div>
```

Classes responsivas

```
<!-- sm: >=576px | md: >=768px | lg: >=992px | xl: >=1200px -->
<div class="row">
  <div class="col-12 col-md-6 col-lg-4">
    Em telas pequenas: 100%
    Em telas médias: 50%
    Em telas grandes: 33.33%
  </div>
</div>
```

Tailwind CSS

Diferente do Bootstrap, o Tailwind CSS é um framework utilitário. Em vez de classes prontas como "card", você combina classes atômicas para construir o layout.

```
<!-- Tailwind CSS -->
<div class="grid grid-cols-1 md:grid-cols-3 gap-4 p-4">
  <div class="bg-blue-500 text-white p-6 rounded-lg">
    Card 1
  </div>
  <div class="bg-green-500 text-white p-6 rounded-lg">
    Card 2
  </div>
  <div class="bg-red-500 text-white p-6 rounded-lg">
    Card 3
  </div>
```

```
</div>
```

Flexbox

O Flexbox é um modelo de layout unidimensional que permite distribuir espaço entre itens em um container. Ambos Bootstrap e Tailwind utilizam Flexbox internamente.

```
/* CSS Flexbox puro */
.container {
  display: flex;
  justify-content: space-between;
  align-items: center;
}

.item {
  flex: 1;
  margin: 0 10px;
}
```

Dica: Escolha o framework CSS que melhor se adapta ao seu projeto. Bootstrap é mais opinativo e oferece componentes prontos. Tailwind oferece mais flexibilidade e controle fino sobre o design.

2.2 Componentes CSS

Os frameworks CSS oferecem uma variedade de componentes prontos que podem ser utilizados diretamente no HTML, economizando tempo de desenvolvimento.

Componentes do Bootstrap

O Bootstrap inclui dezenas de componentes como navbar, cards, formulários, tabelas, modais, carrosséis e muito mais.

Navbar

```
<nav class="navbar navbar-expand-lg navbar-dark bg-dark">
  <div class="container">
    <a class="navbar-brand" href="#">Meu Site</a>
    <button class="navbar-toggler" type="button" data-bs-toggle="collapse"
      data-bs-target="#menu">
      <span class="navbar-toggler-icon"></span>
    </button>
    <div class="collapse navbar-collapse" id="menu">
      <ul class="navbar-nav ms-auto">
        <li class="nav-item"><a class="nav-link" href="#">Home</a></li>
        <li class="nav-item"><a class="nav-link" href="#">Sobre</a></li>
        <li class="nav-item"><a class="nav-link" href="#">Contato</a></li>
      </ul>
    </div>
  </div>
</nav>
```

Cards

```
<div class="card" style="width: 18rem;">
  
  <div class="card-body">
    <h5 class="card-title">Título do Card</h5>
    <p class="card-text">Descrição do card.</p>
    <a href="#" class="btn btn-primary">Saiba mais</a>
  </div>
</div>
```

Formulários

```
<form>
  <div class="mb-3">
    <label for="email" class="form-label">E-mail</label>
    <input type="email" class="form-control" id="email"
      placeholder="seu@email.com">
  </div>
  <div class="mb-3">
    <label for="senha" class="form-label">Senha</label>
    <input type="password" class="form-control" id="senha">
  </div>
  <button type="submit" class="btn btn-primary">Entrar</button>
</form>
```

Tabelas

```
<table class="table table-striped table-hover">
  <thead>
```

```
<tr>
  <th>#</th>
  <th>Nome</th>
  <th>E-mail</th>
</tr>
</thead>
<tbody>
  <tr>
    <td>1</td>
    <td>Maria Silva</td>
    <td>maria@email.com</td>
  </tr>
  <tr>
    <td>2</td>
    <td>João Santos</td>
    <td>joao@email.com</td>
  </tr>
</tbody>
</table>
```

2.3 Ícones

Ícones são elementos visuais que comunicam ideias de forma rápida e intuitiva. Bibliotecas de ícones podem ser facilmente integradas aos projetos web.

Bootstrap Icons

```
<!-- Instalação via CDN -->
<link rel="stylesheet"
      href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.0/font/bootstrap-
      icons.css">

<!-- Uso -->
<i class="bi bi-heart"></i>
<i class="bi bi-star-fill"></i>
<i class="bi bi-person-circle"></i>
<i class="bi bi-gear"></i>
```

React Icons

```
// Instalação
// npm install react-icons

import { FaHeart, FaStar, FaUser } from 'react-icons/fa';
import { FiSettings } from 'react-icons/fi';

function MeuComponente() {
  return (
    <div>
      <FaHeart size={24} color="red" />
      <FaStar size={24} color="gold" />
      <FaUser size={24} />
      <FiSettings size={24} />
    </div>
  );
}
```

Ícones com Tailwind

```
<!-- Heroicons (gratuito e integrado ao Tailwind) -->
<svg class="w-6 h-6 text-gray-500" fill="none" stroke="currentColor"
      viewBox="0 0 24 24">
  <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2"
        d="M4.318 6.318a4.5 4.5 0 00 6.364 12 20.364 17.682 7.682a4.5 4.5 0 00-
        6.364-6.364 12 7.636 1-1.318-1.318a4.5 4.5 0 00-6.364 0z" />
</svg>
```

Dica: Sempre prefira ícones de bibliotecas consolidadas e com boa documentação. Evite criar seus próprios ícones a menos que seja estritamente necessário.

2.4 Customização

Uma das vantagens dos frameworks CSS é a possibilidade de customização, permitindo que o desenvolvedor adapte o estilo à identidade visual do projeto.

Customização no Bootstrap

O Bootstrap utiliza variáveis CSS e SASS para permitir a personalização de cores, fontes, espaçamentos e outros aspectos.

```
/* Personalizando variáveis CSS do Bootstrap */
:root {
  --bs-primary: #ff6600;
  --bs-primary-rgb: 255, 102, 0;
  --bs-border-radius: 0.5rem;
}

/* Ou compilando SASS */
$primary: #ff6600;
$border-radius: 0.5rem;
@import "bootstrap";
```

Customização no Tailwind

O Tailwind é altamente customizável através do arquivo de configuração tailwind.config.js.

```
// tailwind.config.js
module.exports = {
  theme: {
    extend: {
      colors: {
        primary: '#ff6600',
        secondary: '#1a365d',
      },
      fontFamily: {
        heading: ['Montserrat', 'sans-serif'],
      },
      spacing: {
        '18': '4.5rem',
        '22': '5.5rem',
      },
    },
  },
}
```

Temas claro/escuro

```
/* Usando variáveis CSS para temas */
:root {
  --bg-color: #ffffff;
  --text-color: #333333;
}

[data-theme="dark"] {
  --bg-color: #1a1a1a;
  --text-color: #f0f0f0;
}

body {
```

```
background-color: var(--bg-color);
color: var(--text-color);
}
```

Personalizando componentes Bootstrap com React

```
// React + Bootstrap customizado
function BotaoPersonalizado({ variant, children }) {
  const classes = `btn btn-${variant} rounded-pill px-4`;
  return <button className={classes}>{children}</button>
}

<BotaoPersonalizado variant="primary">Clique aqui</BotaoPersonalizado>
```

Dica: Ao customizar um framework CSS, mantenha a consistência visual. Defina um guia de estilos com as cores, fontes e espaçamentos do projeto antes de começar a codificar.

Capítulo 3 – Práticas de Desenvolvimento de Sistemas

Neste capítulo, abordaremos práticas essenciais para o desenvolvimento profissional de sistemas, incluindo testes automatizados, processos de build e release, e controle de versão.

3.1 Testes unitários

Testes unitários são testes automatizados que verificam o funcionamento de unidades isoladas do código, como funções, métodos ou componentes.

Por que testar?

- Detectar erros precocemente no desenvolvimento.
- Facilitar a refatoração e manutenção do código.
- Documentar o comportamento esperado do sistema.
- Aumentar a confiança na entrega de novas funcionalidades.

Jest

Jest é um framework de testes JavaScript amplamente utilizado, desenvolvido pelo Facebook. Ele funciona "zero config" na maioria dos projetos React.

Instalação

```
npm install --save-dev jest @testing-library/react
```

Testando funções

```
// funcoes.js
export function somar(a, b) {
  return a + b;
}

export function ehPar(numero) {
  return numero % 2 === 0;
}

// funcoes.test.js
import { somar, ehPar } from './funcoes';

describe('Funções matemáticas', () => {
  test('somar deve retornar a soma correta', () => {
    expect(somar(2, 3)).toBe(5);
    expect(somar(-1, 1)).toBe(0);
    expect(somar(0, 0)).toBe(0);
  });
});

describe('Função ehPar', () => {
  test('deve retornar true para números pares', () => {
    expect(ehPar(2)).toBe(true);
    expect(ehPar(0)).toBe(true);
  });

  test('deve retornar false para números ímpares', () => {
    expect(ehPar(3)).toBe(false);
    expect(ehPar(7)).toBe(false);
  });
});
```

```
});
```

Testando componentes React

```
// Saudacao.jsx
function Saudacao({ nome }) {
  return <h1>Olá, {nome}</h1>;
}

export default Saudacao;

// Saudacao.test.jsx
import { render, screen } from '@testing-library/react';
import Saudacao from './Saudacao';

describe('Componente Saudacao', () => {
  test('deve renderizar a saudação com o nome', () => {
    render(<Saudacao nome="Maria" />);
    expect(screen.getByText('Olá, Maria!')).toBeInTheDocument();
  });

  test('deve renderizar "Olá, !" quando nome é vazio', () => {
    render(<Saudacao nome="" />);
    expect(screen.getByText('Olá, !')).toBeInTheDocument();
  });
});
```

Testando eventos

```
// Contador.jsx
function Contador() {
  const [valor, setValor] = useState(0);
  return (
    <div>
      <p>Valor: {valor}</p>
      <button onClick={() => setValor(valor + 1)}>Incrementar</button>
    </div>
  );
}

// Contador.test.jsx
import { render, screen, fireEvent } from '@testing-library/react';
import Contador from './Contador';

test('deve incrementar o valor ao clicar no botão', () => {
  render(<Contador />);
  const botao = screen.getByText('Incrementar');
  fireEvent.click(botao);
  expect(screen.getByText('Valor: 1')).toBeInTheDocument();
});
```

Dica: Siga o padrão AAA (Arrange, Act, Assert) nos seus testes: organize o cenário, execute a ação e verifique o resultado.

3.2 Testes de interface

Testes de interface (ou testes end-to-end) verificam o funcionamento da aplicação do ponto de vista do usuário, simulando interações reais em um navegador.

O que são testes E2E?

Diferente dos testes unitários, que testam partes isoladas, os testes end-to-end testam o sistema completo, incluindo front-end, back-end e banco de dados, simulando cenários reais de uso.

Cypress

Cypress é uma ferramenta moderna para testes de interface, com uma interface gráfica poderosa e comandos intuitivos.

Instalação

```
npm install --save-dev cypress
```

Exemplo de teste Cypress

```
// cypress/e2e/login.cy.js
describe('Página de Login', () => {
  beforeEach(() => {
    cy.visit('/login');
  });

  it('deve exibir o formulário de login', () => {
    cy.get('input[type="email"]').should('be.visible');
    cy.get('input[type="password"]').should('be.visible');
    cy.get('button[type="submit"]').should('contain', 'Entrar');
  });

  it('deve exibir erro para credenciais inválidas', () => {
    cy.get('input[type="email"]').type('invalido@email.com');
    cy.get('input[type="password"]').type('senha_errada');
    cy.get('button[type="submit"]').click();
    cy.contains('Credenciais inválidas').should('be.visible');
  });

  it('deve redirecionar ao fazer login com sucesso', () => {
    cy.get('input[type="email"]').type('admin@email.com');
    cy.get('input[type="password"]').type('123456');
    cy.get('button[type="submit"]').click();
    cy.url().should('include', '/dashboard');
  });
});
```

Testando uma lista de tarefas

```
// cypress/e2e/tarefas.cy.js
describe('Lista de Tarefas', () => {
  it('deve permitir adicionar e remover tarefas', () => {
    cy.visit('/tarefas');

    // Adicionar tarefa
    cy.get('input[placeholder="Nova tarefa"]').type('Estudar Cypress');
    cy.get('button').contains('Adicionar').click();
    cy.contains('Estudar Cypress').should('be.visible');
  });
});
```

```

    // Marcar como concluída
    cy.get('input[type="checkbox"]').first().check();
    cy.contains('Estudar Cypress').should('have.class', 'concluida');

    // Remover tarefa
    cy.get('button').contains('Remover').click();
    cy.contains('Estudar Cypress').should('not.exist');
  });
});

```

Testes de interface com React Testing Library

```

import { render, screen, waitFor } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import App from './App';

test('navegação completa do usuário', async () => {
  render(<App />);

  // Simular digitação
  await userEvent.type(screen.getByLabelText('Nome'), 'Maria');
  await userEvent.type(screen.getByLabelText('E-mail'), 'maria@email.com');

  // Clicar no botão de envio
  await userEvent.click(screen.getByRole('button', { name: 'Enviar' }));

  // Verificar resultado
  await waitFor(() => {
    expect(screen.getByText('Cadastro realizado!')).toBeInTheDocument();
  });
});

```

Dica: Prefira testar comportamentos visíveis ao usuário em vez de detalhes de implementação. Isso torna os testes mais robustos a refatorações.

3.3 Builds e releases

O processo de build transforma o código fonte em um conjunto de arquivos otimizados prontos para produção. Release é a entrega desses arquivos aos usuários.

O que é build?

Build é o processo de compilação e empacotamento dos arquivos do projeto. Envolve etapas como transpilação (JSX para JavaScript), minificação, tree shaking, code splitting e otimização de assets.

Build no React (Vite)

```
// Comando de build
npm run build

// Resultado: pasta dist/ com arquivos otimizados
dist/
  index.html
  assets/
    index-abc123.js  # JS minificado e ofuscado
    index-xyz456.css # CSS minificado
    logo-789def.svg  # Imagens otimizadas
```

Configuração de build (Vite)

```
// vite.config.js
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
  build: {
    outDir: 'dist',
    minify: 'terser',
    sourcemap: false,
    rollupOptions: {
      output: {
        manualChunks: {
          vendor: ['react', 'react-dom'],
        },
      },
    },
  },
});
```

Processo de release

Um release bem estruturado segue etapas definidas:

- 1. Desenvolvimento e testes em ambiente local.
- 2. Integração contínua (CI): execução automática de testes.
- 3. Build de produção com otimizações.
- 4. Deploy em ambiente de homologação/staging.
- 5. Testes de aceitação no ambiente de staging.
- 6. Deploy em produção.

CI/CD

```
# Exemplo de pipeline CI/CD (GitHub Actions)
# .github/workflows/deploy.yml
name: Deploy

on:
  push:
    branches: [main]

jobs:
  build-and-deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - uses: actions/setup-node@v3
        with:
          node-version: 18
      - run: npm ci
      - run: npm test
      - run: npm run build
      - uses: peaceiris/actions-gh-pages@v3
        with:
          github_token: ${ secrets.GITHUB_TOKEN }
          publish_dir: ./dist
```

Deploy em plataformas populares

Algumas plataformas populares para deploy de aplicações React:

- Vercel: deploy simples conectado ao GitHub.
- Netlify: similar ao Vercel, com suporte a funções serverless.
- GitHub Pages: ideal para projetos estáticos e portfólios.
- Firebase Hosting: integrado com outros serviços Google.

Dica: Sempre teste o build localmente antes de fazer o deploy. Use o comando `npm run build` e verifique se não há erros ou warnings.

3.4 Controle de versão

Controle de versão é um sistema que registra todas as alterações feitas em um conjunto de arquivos ao longo do tempo. Ele permite recuperar versões anteriores, colaborar com outros desenvolvedores e gerenciar diferentes linhas de desenvolvimento.

Git

Git é o sistema de controle de versão mais utilizado atualmente. Ele é distribuído, ou seja, cada desenvolvedor tem uma cópia completa do repositório.

Comandos fundamentais

```
# Configuração inicial
git config --global user.name "Seu Nome"
git config --global user.email "seu@email.com"

# Criar um repositório
git init                                # Inicia repositório local
git clone https://github.com/usuario/projeto.git # Clona repositório remoto

# Alterações
git status                                # Verifica status dos arquivos
git add arquivo.js                        # Adiciona arquivo à staging area
git add .                                  # Adiciona todos os arquivos
git commit -m "Mensagem do commit"        # Cria um commit

# Histórico
git log                                    # Exibe histórico de commits
git log --oneline                          # Histórico resumido
git diff                                    # Mostra alterações não commitadas

# Branches
git branch                                # Lista branches
git branch feature-login                  # Cria nova branch
git checkout feature-login                # Muda para a branch
git checkout -b feature-login              # Cria e muda para nova branch
git merge feature-login                    # Faz merge na branch atual
```

Fluxo de trabalho com branches

O fluxo de trabalho com branches permite desenvolver funcionalidades em paralelo sem interferir no código principal.

```
# Fluxo básico: Git Flow simplificado
# 1. Criar branch para nova funcionalidade
git checkout -b feature/nova-funcionalidade

# 2. Desenvolver e commitar
git add .
git commit -m "Implementa nova funcionalidade"

# 3. Voltar para main e atualizar
git checkout main
git pull origin main

# 4. Integrar a funcionalidade
git merge feature/nova-funcionalidade
```

```
# 5. Enviar para o repositório remoto
git push origin main
```

Plataformas de hospedagem

As plataformas de hospedagem de repositórios Git permitem colaboração, code review e integração com ferramentas de CI/CD.

- GitHub: plataforma mais popular, com issues, pull requests e Actions.
- GitLab: oferece CI/CD integrado e auto-hospedagem.
- Azure DevOps: integrado ao ecossistema Microsoft.

.gitignore

O arquivo **.gitignore** lista arquivos e diretórios que o Git deve ignorar, como node_modules, arquivos de build e configurações locais.

```
# .gitignore
node_modules/
dist/
.env
*.log
.DS_Store
.vscode/
```

Boas práticas

- Faça commits pequenos e com mensagens descritivas.
- Nunca commite arquivos de configuração com senhas ou chaves de API.
- Sempre puxe as alterações (pull) antes de começar a trabalhar.
- Resolva conflitos localmente antes de fazer push.
- Use branches para desenvolver novas funcionalidades.
- Faça code reviews através de pull requests.

Dica: *Mantenha commits atômicos: cada commit deve representar uma única mudança lógica. Isso facilita a revisão, o rollback e a identificação de problemas.*

Exercícios de Fixação

Os exercícios a seguir têm o objetivo de fixar os conceitos apresentados nesta apostila. Resolva cada um deles utilizando os conhecimentos adquiridos.

Capítulo 1

1. Crie um componente React chamado "Saudacao" que recebe uma prop "nome" e exibe "Olá, [nome]!". Em seguida, utilize esse componente três vezes com nomes diferentes.
2. Desenvolva um contador com dois botões: um para incrementar e outro para decrementar. O valor nunca deve ser negativo.
3. Crie uma lista de tarefas onde o usuário pode adicionar novas tarefas e marcá-las como concluídas.
4. Implemente um formulário de cadastro (nome, e-mail, idade) com validação básica de campos obrigatórios.
5. Crie três componentes: Cabeçalho, Conteúdo e Rodapé. Compose-os em uma página utilizando o conceito de composição de componentes.

Capítulo 2

1. Crie um layout responsivo com Bootstrap contendo: cabeçalho, três colunas de conteúdo e rodapé. O layout deve se adaptar para uma coluna em telas pequenas.
2. Utilizando Tailwind CSS, reproduza um card de produto com imagem, título, descrição e botão "Comprar".
3. Crie uma navbar responsiva com Bootstrap que colapse em dispositivos móveis.
4. Adicione ícones do React Icons a uma lista de contatos (telefone, e-mail, localização).
5. Customize o tema do Bootstrap alterando as cores primárias e o arredondamento das bordas.

Capítulo 3

1. Escreva testes unitários para uma função que valida CPF (apenas a lógica de validação, sem formatação).
2. Crie um teste de componente React que verifica se um botão desabilita quando um campo está vazio.
3. Utilizando Cypress, escreva um teste que visita uma página de login, preenche os campos e submete o formulário.
4. Configure um pipeline básico de CI/CD no GitHub Actions que executa os testes e faz o build do projeto.
5. Crie um repositório Git local, faça commits em branches diferentes e realize um merge.

Referências Bibliográficas

- React Documentation. Disponível em: <https://react.dev>
- Bootstrap Documentation. Disponível em: <https://getbootstrap.com/docs>
- Tailwind CSS Documentation. Disponível em: <https://tailwindcss.com/docs>
- Jest Documentation. Disponível em: <https://jestjs.io/docs>
- Cypress Documentation. Disponível em: <https://docs.cypress.io>
- React Router Documentation. Disponível em: <https://reactrouter.com>
- Git Documentation. Disponível em: <https://git-scm.com/doc>
- Vite Documentation. Disponível em: <https://vitejs.dev>